Programming for Robotics: Project Report

ATLAS: Autonomous Tracking Laser Aiming System

Author 1:	Madeline Abio s5259572
Author 2:	Alpa Sahai s5405889
GitHub Link:	https://github.com/alpasahai/programming_robotics_project.git
Specialisation Option Chosen:	Embedded Intelligence

Griffith University

Contents

List of Figures

1. Project Overview

The Autonomous Tracking and Laser Aiming System (ATLAS) is a simulated fire-control system that detects, tracks, and neutralises airborne threats without human input, operating in an outdoor Webots environment. It is split into two collaborative units: a Search Radar that sweeps a wide area with a wide-beam radar sensor, and a turret-mounted Fire-Control Radar (FCR) carrying a narrow-beam radar and projectile launcher. The two units coordinate over emitter/receiver pairs, with rotational motors driving the search sweep and the FCR's pan/tilt turret.

When the Search Radar spots a target, it cues the FCR, which locks on and fuses readings from both radars through a Kalman filter to smooth noisy measurements and predict the target's trajectory before firing. Alongside this, an online SGDClassifier learns the attacker's launch-direction pattern live via `partial_fit`, pre-slewing the turret toward the predicted next sector while idle and re-adapting continuously as the pattern drifts.

2. Technical Requirements Compliance

This section maps ATLAS to each technical requirement; details live in the cited sections. The figure and `FSM` label are defined here once and referenced elsewhere.

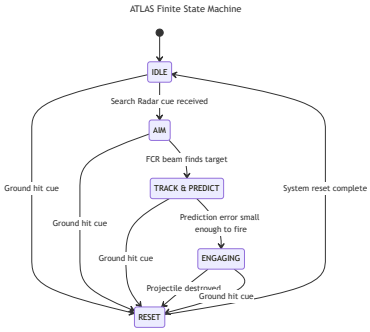

Figure 1: ATLAS finite state machine (FSM) diagram

- **Inputs ($\geq$ 2).** Search Radar cue (world-frame, via radio) and Fire Control Radar position (turret-relative, when locked). Full list and noise figures in Section 5.
- **Outputs ($\geq$ 2).** Pan/tilt motor `setPosition()` commands and a bullet launch (teleport-to-muzzle plus velocity write). Detailed in Section 3.
- **FSM with $\geq$ 4 states.** Five states — `IDLE`, `AIM`, `TRACK_PREDICT`, `ENGAGING`, `RESET` (Figure 1, Section 4).
- **Multi-condition decision logic.** `TRACK_PREDICT` $\rightarrow$ `ENGAGING` requires prediction error below threshold AND intercept in range AND sustained for N frames; FCR lock combines FOV-cone membership AND range. See Section 5.
- **Safety / fail-safe.** Convergence gate before firing, universal `RESET` recovery, and the anti-friendly-fire pan-angle gate. Detailed in Section 6.
- **Sensor fusion.** Heterogeneous Kalman fusion of the Search Radar ($R_{\text{SEARCH}} = 0.1$) and FCR ($R_{\text{FCR}} = 0.001$), with predict-every-tick for dropout safety. See Section 5.
- **Context-aware behaviour.** The `AttackPredictor` learns launch-sector patterns online and pre-slews the turret in `IDLE`; the convergence gate adapts to track volatility. See Section 4.
- **Real-time logic.** Synchronous Sense $\rightarrow$ Think $\rightarrow$ Act loop on the 32 ms Webots tick, with sensing and FSM evaluation running every tick. See Section 3.
- **Advanced AI / adaptive component.** Online logistic regression (`SGDClassifier`, `partial_fit` per launch) over online-discovered sectors with EMA drift tracking — one learning step and one next-sector prediction per engagement. See Section 4.

3. System Architecture

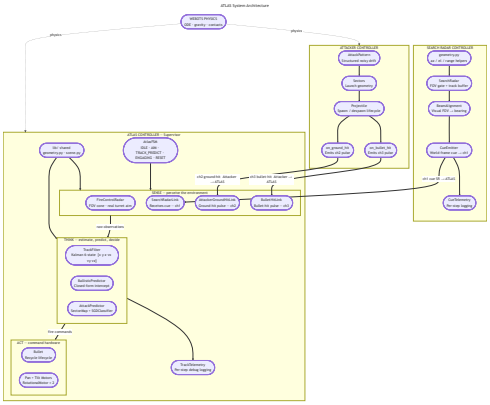

Figure 2: ATLAS system architecture diagram

Every 32 ms tick the ATLAS controller runs the three-stage Sense → Think → Act loop in order; Figure 2 groups its components by stage, with arrows showing the data that crosses each boundary. Information flows in one direction each tick: raw sensor reads → world snapshot (Sense) → filtered state + chosen action (Think) → motor and bullet writes (Act).

Sense reads the Webots devices — radio receivers, the on-board FCR, and the pan/tilt joint-angle sensors — and emits a fixed-shape world snapshot for the tick: a Search Radar cue (or none), an FCR position (when locked), any hit pulses, and the measured boresight. No interpretation happens here. Noise figures and per-stream behaviour are in Section 5.

Think takes the snapshot and produces a chosen action through three activities: **estimation** (the Kalman `TrackFilter` maintains a fused position-velocity estimate), **prediction** (the `BallisticPredictor` extrapolates an intercept and the `AttackPredictor` learns launch-sector patterns online), and **decision** (the `AtlasFSM` arbitrates via guarded transitions over those products). The output is a target pan/tilt and a fire flag. The state machine is detailed in Section 4.

Act turns that decision into hardware writes: `setPosition()` on the pan and tilt motors, and — when the FSM commits to a shot — teleport plus velocity writes on the recycled bullet `Solid`. A pan-angle friendly-fire gate sits between the FSM and the bullet to suppress shots that would cross the Search Radar's bearing.

4. Behavioural Model

ATLAS uses an FSM for autonomous target detection, tracking, prediction, and engagement. FSM control gives deterministic behaviour, modular transitions, and a guaranteed fail-safe recovery path under noisy sensing. The five states are summarised below; Figure 1 shows the full transition diagram. Sensor processing details (the Kalman filter, fusion weighting, and convergence gate) are covered in Section 5 and are referenced rather than repeated here.

`IDLE`. The turret holds its search beam, or pre-aims at the `AttackPredictor`'s next predicted sector, while waiting for a Search Radar cue. On the first cue of a new launch, the `AttackPredictor` performs one online learning step on the just-observed launch and immediately predicts the sector after that, which `IDLE` consumes for pre-aim. The FSM moves to `AIM` once a cue has been present for several consecutive frames, or to `RESET` on a ground-hit cue.

`AIM`. The turret slews toward the raw Search Radar cue; the Kalman estimate has just been reset and is not yet trustworthy, so no fusion runs. The FSM moves to `TRACK_PREDICT` as soon as the FCR locks on, or to `RESET` on a ground-hit cue.

`TRACK_PREDICT`. Each tick the turret slews to the latest filtered position, keeping the projectile inside the FCR's FOV so it returns fresh measurements — closing a tight track-and-aim loop. The filter fuses those measurements with the Search Radar cue, and the `BallisticPredictor` propagates the estimate forward to an intercept. The FSM moves to `ENGAGING` once the convergence gate trips, or to `RESET` on a ground-hit cue.

ENGAGING. The turret holds aim at the frozen intercept and fires a bullet, subject to the anti-friendly-fire pan-angle gate. No further prediction runs. The FSM moves to `RESET` on a bullet-hit cue, a ground-hit cue, or the target leaving range.

RESET. Wipes the Kalman state, clears the stale Search Radar cue, and resets the prediction history and intercept so no stale data leaks into the next engagement. The FSM then returns to `IDLE`.

5. Perception / Sensor Processing

Perception is built from radar, inter-process radio, and joint-angle sensors. Each tick the controller drains five streams: a coarse world-frame cue from the Search Radar (noise std ≈ 0.2 m), a turret-relative position from the on-board Fire-Control Radar (FCR) when locked (noise std ≈ 0.02 m), one-shot ground-hit and bullet-hit pulses, and the pan/tilt joint angles giving the measured boresight.

At the core of the pipeline is a Kalman filter — a running estimate of the projectile's position (x, y, z) and velocity (v_x, v_y, v_z). The **predict** step advances the estimate each tick using projectile physics (position by velocity, gravity on velocity), so a transient dropout cannot collapse the track. The **update** step blends incoming measurements with the prediction; radars report position only, so velocity is inferred from its evolution over time. Measurement weight is governed by the noise variance R, and the two radars are separated by two orders of magnitude — $R_{\text{FCR}} = 0.001$ vs $R_{\text{SEARCH}} = 0.1$ — so the FCR dominates corrections. The FCR is fused on every locked tick; the Search Radar is fused only during `TRACK_PREDICT`, adding a weak long-range correction without polluting the precise track. The resulting estimate feeds the `BallisticTrajectoryPredictor`, which extrapolates to a closed-form intercept.

These products drive the FSM through compound guards. The FCR lock flag promotes `AIM` → `TRACK_PREDICT`; the fire decision is stricter, requiring the predicted-vs-observed error to stay below threshold AND the intercept to stay in range, both for several consecutive frames, before `TRACK_PREDICT` → `ENGAGING`. A final gate suppresses the shot if the measured pan angle lies inside the friendly-radar exclusion cone.

6. Safety and Robustness

ATLAS layers several safety mechanisms over the FSM and perception pipeline. The two principal ones are:

Convergence gate (fail-safe before firing). `TRACK_PREDICT` → `ENGAGING` only trips once the predicted-vs-observed error stays below threshold AND the intercept stays in range for several consecutive ticks, so a single-frame "good prediction" cannot commit the turret to a shot.

Anti-friendly-fire exclusion zone. Every fire command is gated against the measured pan angle; if the boresight lies within ± 0.4 rad ($\approx 23°$) of the Search Radar's bearing, the shot is suppressed. The turret may still track through the cone — only firing is blocked.

Supporting measures: universal `RESET` recovery (every state can exit to `RESET`, which wipes filter state before returning to `IDLE`), sensor-dropout tolerance (the Kalman predict step runs every tick regardless of measurement availability), and a debounced `IDLE` → `AIM` transition that rejects single-frame cue flickers.

7. Code Structure

`atlas_controller.py` is the hub from which all of ATLAS's system logic is delegated. It is the only module inside the turret package that talks to Webots directly — every other module operates on plain Python objects passed in by the controller. At startup it instantiates every module, wires their dependencies, and then loops on `robot.step(timestep)` until the simulation ends. The loop body is deliberately thin — no control logic lives in the controller itself; it only delegates, in a fixed order, to the modules below and applies whatever commands they emit. The other Webots process, `search_radar_controller/`, is comparatively simple: it sweeps a beam and emits a noisy world-frame cue over radio, which ATLAS consumes as one of its inputs.

ATLAS's modules each carry a single responsibility. `FireControlRadar` is the on-board radar; `SearchRadarLink`, `AttackerGroundHitLink`, and `BulletHitLink` are the radio receivers for the Search Radar cue and the attacker's resolution pulses. `TrackFilter` is the Kalman estimator (built on `filterpy`), `BallisticTrajectoryPredictor` computes the intercept, and `AttackPredictor` (with its supporting

`SectorMap` and `LaunchLatch`) is the online learner. `AtlasFSM` is the state machine. `Bullet` owns the recycled-projectile lifecycle. `telemetry.py` and `scoreboard.py` are observers and have no effect on control.

Third-party libraries: `numpy` (linear algebra), `filterpy` (Kalman), `scikit-learn` (`SGDClassifier` for online learning), and the Webots `controller` API. The modules are written against stub interfaces, so the 26 `pytest` files under each controller's `tests/` directory run without Webots.

8. Key Code Snippets

```python
# fsm.py — TRACK_PREDICT → ENGAGING convergence gate
intercept = self.sensors.ballistic_predictor.get_intercept(
    self.config.lookahead_steps
)
pred_error = self._record_intercept_and_error(intercept, position)

if (
    pred_error is not None
    and pred_error < self.config.track_error_threshold
    and self._target_within_range(intercept)
):
    self._converge_count += 1
else:
    self._converge_count = 0

if self._converge_count >= self.config.converge_frames:
    self._intercept = intercept
    self._transition(self.ENGAGING)
```

Listing 1: Convergence gate: fires only on sustained low prediction error AND in-range intercept.

```python
# track_filter.py — heterogeneous Kalman fusion
def predict(self) -> None:
    u = np.array([[0.0], [0.0], [_GRAVITY]])
    self.kf.predict(u=u)

def update_fcr(self, measurement):
    self.kf.R = np.eye(3) * self._R_fcr  # low noise — trusted
    self.kf.update(np.array(measurement).reshape(3, 1))
    self._initialised = True

def update_search(self, measurement):
    self.kf.R = np.eye(3) * self._R_search  # high noise — weak nudge
    self.kf.update(np.array(measurement).reshape(3, 1))
    self._initialised = True
```

Listing 2: Kalman fusion: gravity is a control input; per-source R is swapped before each update so FCR dominates corrections.

```python
# attack_predictor.py — one learning step + one prediction per launch
def observe(self, bearing, time) -> int:
    sector = self._sector_map.observe(bearing)

    # Online update: previous context → current sector label.
    if self._history:
        X = extract_features(self._history, self._max_sectors).reshape(1, -1)
        y = [sector]
        if not self._fitted:
            self._model.partial_fit(X, y, classes=np.arange(self._max_sectors))
            self._fitted = True
        else:
            self._model.partial_fit(X, y)

    self._history.append(sector)
    self._seen.add(sector)

    # Predict the next sector from the updated context.
    if self._fitted and len(self._seen) >= self._min_sectors_seen:
        X_next = extract_features(self._history, self._max_sectors).reshape(1, -1)
        self._predicted_sector = int(self._model.predict(X_next)[0])
    else:
        self._predicted_sector = None

    return sector
```

Listing 3: Online learning: one `partial_fit` per launch, then predict the next sector.

9. Results / Demonstration Summary

ATLAS successfully demonstrates autonomous projectile detection, tracking, prediction, and interception within the Webots simulation environment. The Search Radar cues the turret with coarse world-frame positions; the on-board FCR locks once the turret has slewed onto the target; the Kalman `TrackFilter` and `BallisticTrajectoryPredictor` then compute an intercept and the turret fires a bullet at it. Real Webots physics determine the outcome — the bullet either strikes the projectile or the projectile reaches the ground. Across the test run, ATLAS shoots down approximately **52%** of incoming projectiles. The remainder fall to ground hits, mostly when the convergence gate hasn't tripped before the projectile leaves the engagement range.

The system's safety and fail-safe behaviours — the convergence gate, universal `RESET` recovery, and the anti-friendly-fire pan-angle gate — all operated as specified in the test run. The main implementation challenges were modelling the Search Radar and FCR as 3D radars in Webots (the built-in radar is 2D only), characterising the projectile's ballistic motion well enough for the intercept pipeline to converge, and building the adaptive attack-pattern learner so that prediction quality improved monotonically across engagements.

10. Individual Contributions

Member	Primary area of ownership
Madeline Abio	FCR and Search Radar controllers; Kalman `TrackFilter` and `BallisticTrajectoryPredictor`; `AtlasFSM` and the multi-condition convergence gate; `AttackPredictor` online learning component.
Alpa Sahai	Webots world file, robot PROTOs, and scene setup; anti-friendly-fire safety exclusion zone; simulating the 3D radars; system architecture diagram; integration testing, report writing, and demo video.

Table 1: Project contributions, with significant overlap via pair programming.